Experiment 8

Aim: Implement the Blockchain platform Ganache

Theory:

Ganache is a personal Ethereum blockchain used for testing and development. It allows developers to deploy contracts, develop decentralized applications (DApps), and run tests in a safe and deterministic environment.

Ganache provides:

- Instant mining of blocks
- Pre-funded test accounts
- Transaction history view
- Graphical interface to monitor blockchain activities

It is part of the Truffle Suite and is commonly used with Remix and Metamask for smart contract deployment.

Steps to Connect Ganache with Metamask and Remix IDE:

1. Install Ganache:
    - Download and install Ganache from the Truffle Suite website.
    - Launch it to start a local blockchain network with test Ether accounts.
2. Connect Ganache Accounts with Metamask:
    - Open Metamask and switch to a Custom RPC Network.
    - Enter the Ganache RPC server URL (e.g., HTTP://127.0.0.1:7545).
    - Import private keys of Ganache accounts into Metamask.
3. Connect Remix IDE with Metamask:
    - Open Remix IDE in the browser.
      Choose Injected Web3 as the environment in the “Deploy & Run Transactions” tab.
    - Remix will automatically connect to the Metamask account (linked with Ganache).
4. Create a Simple Solidity Smart Contract:
    - Write a Solidity program related to your mini project (e.g., Student Record, Donation Tracker, etc.).

```
pragma solidity ^0.8.0;
    contract SimpleStorage {
        uint public data;
        function set(uint x) public {
            data = x;
        }
```

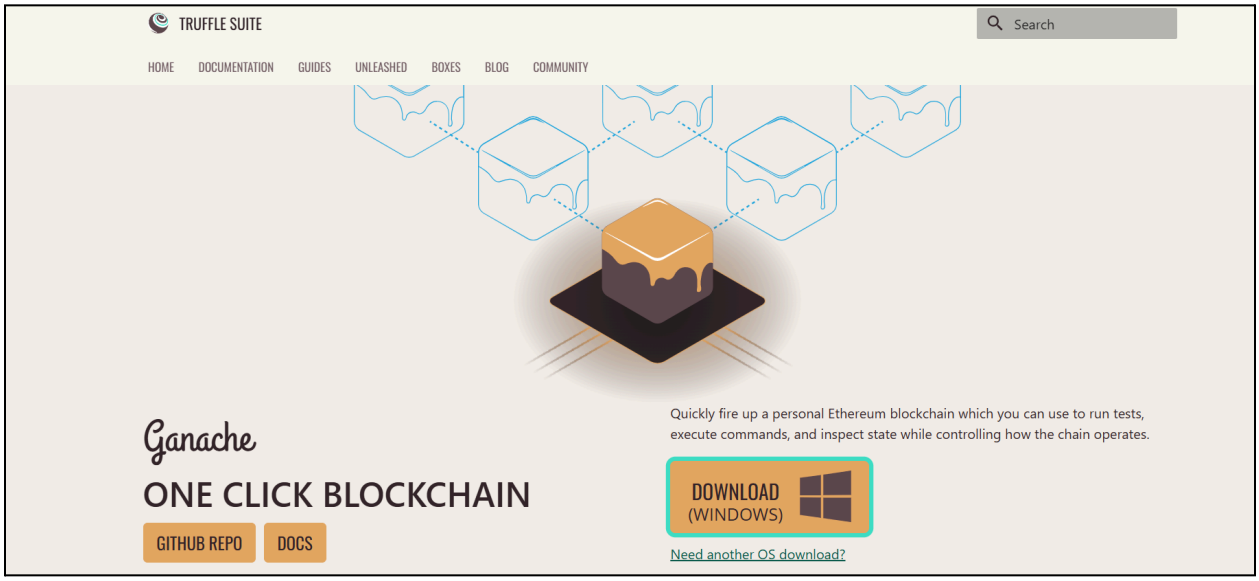
5. Compile and Deploy the Contract:
   Compile using Remix.
   - Deploy via the Metamask account connected to Ganache.
   - Approve the transaction in Metamask.
6. Check Transaction Details on Ganache:
   - View mined transactions, block details, and balances in the Ganache interface.
7. Interact with the Smart Contract:
   - Use Remix to call contract functions (set() or get()).
   - Observe the state changes and corresponding transactions on Ganache.

Outcome:

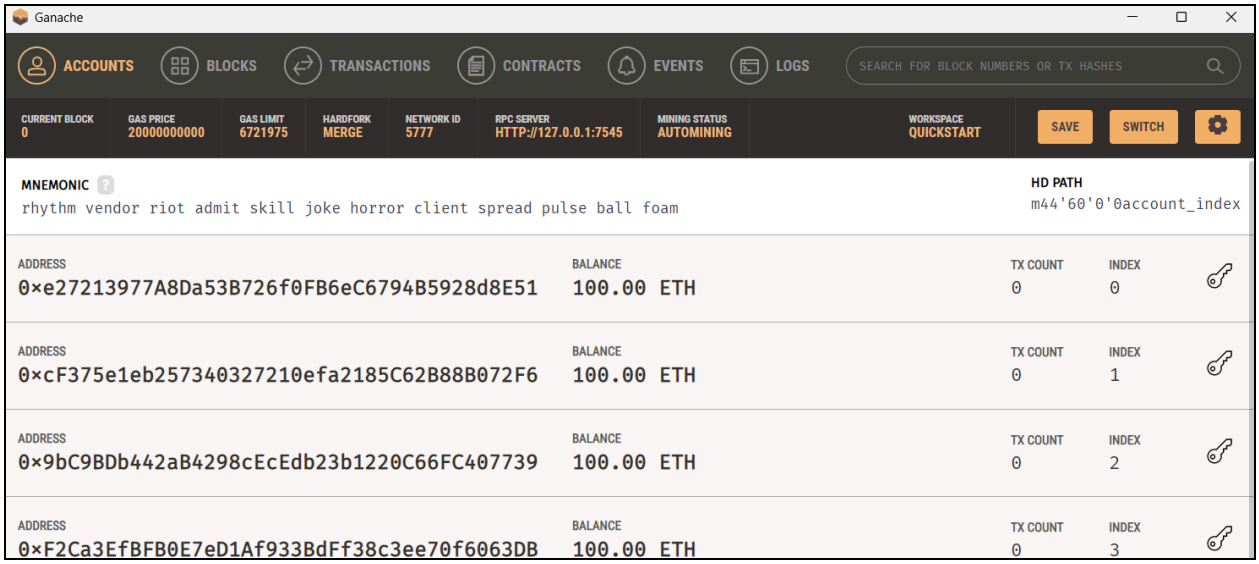
- Understood how to link **Metamask** with **Remix IDE** for Ethereum smart contract deployment.
- Successfully deployed and interacted with a Solidity smart contract through **Ganache**.
- Verified all transactions and balances within the Ganache environment.

Implementation:

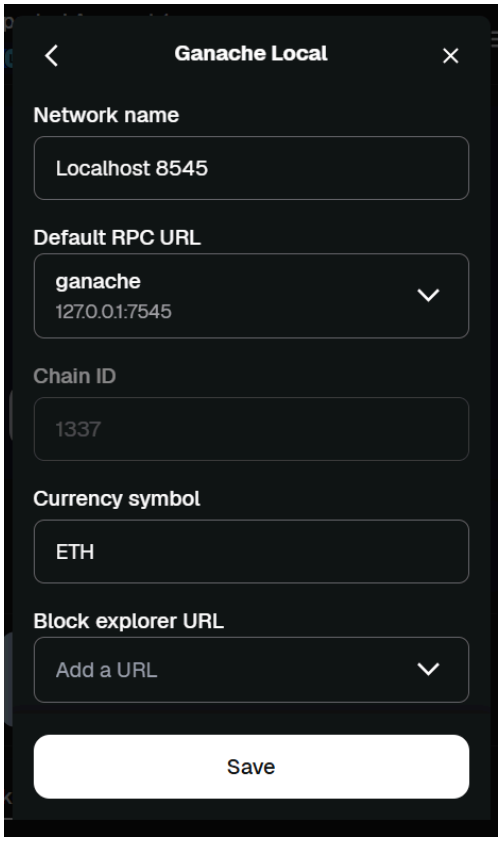
Step 1: Download and install ganache



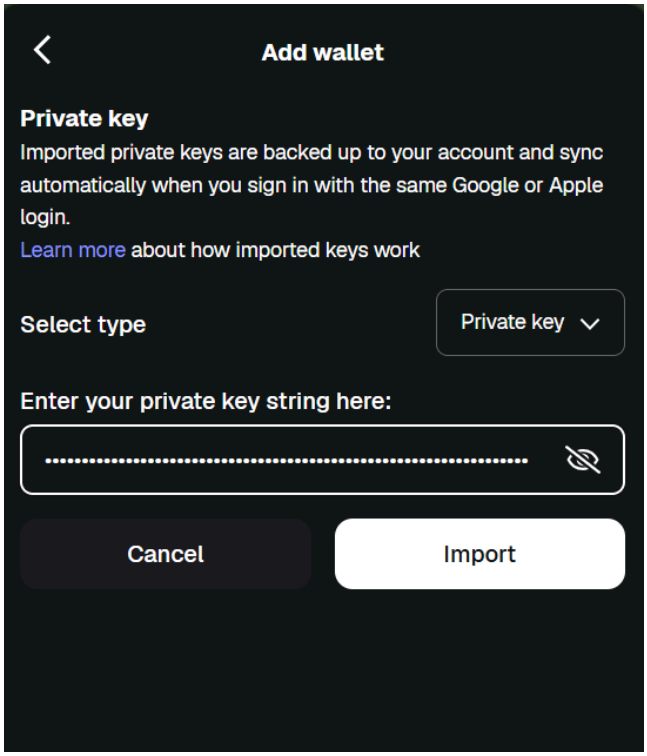
Once installed, open ganache and click on Quickstart Ethereum to start localhost.



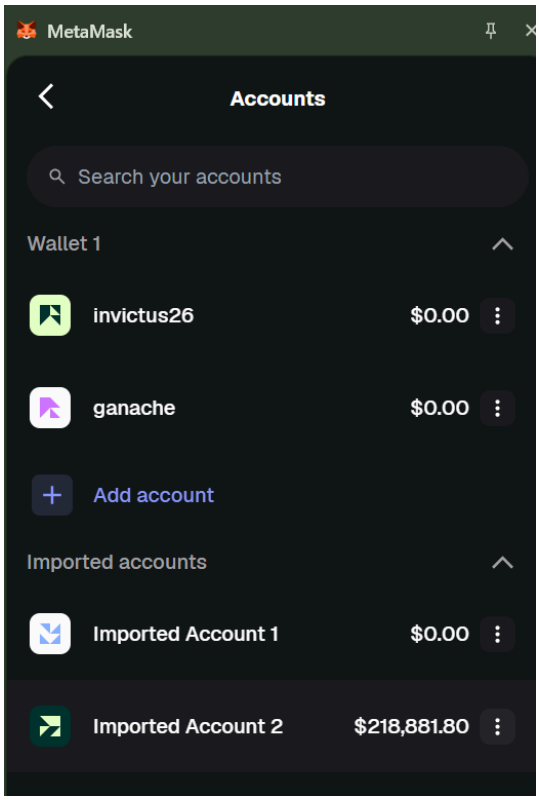
Step 2: In MetaMask, click on the networks tab and add a custom network.

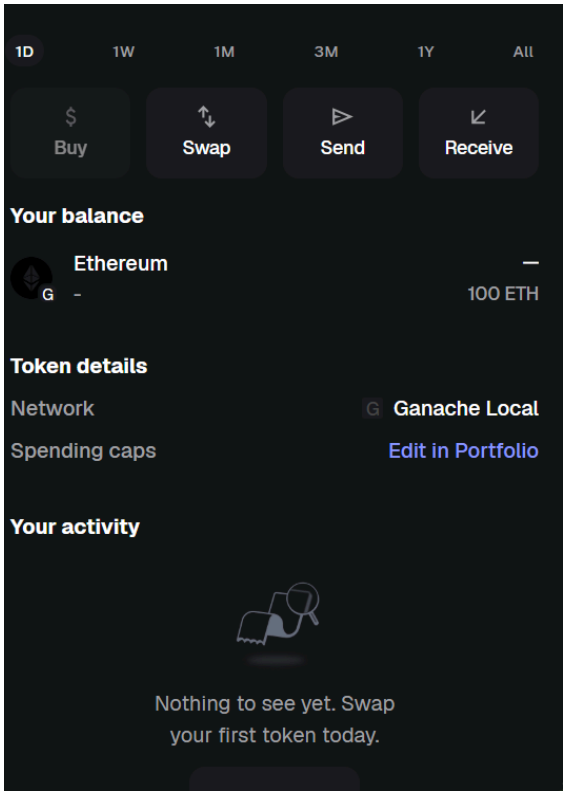


Step 3: Import ganache account into metamask



Wallet balance appears in metaMask account





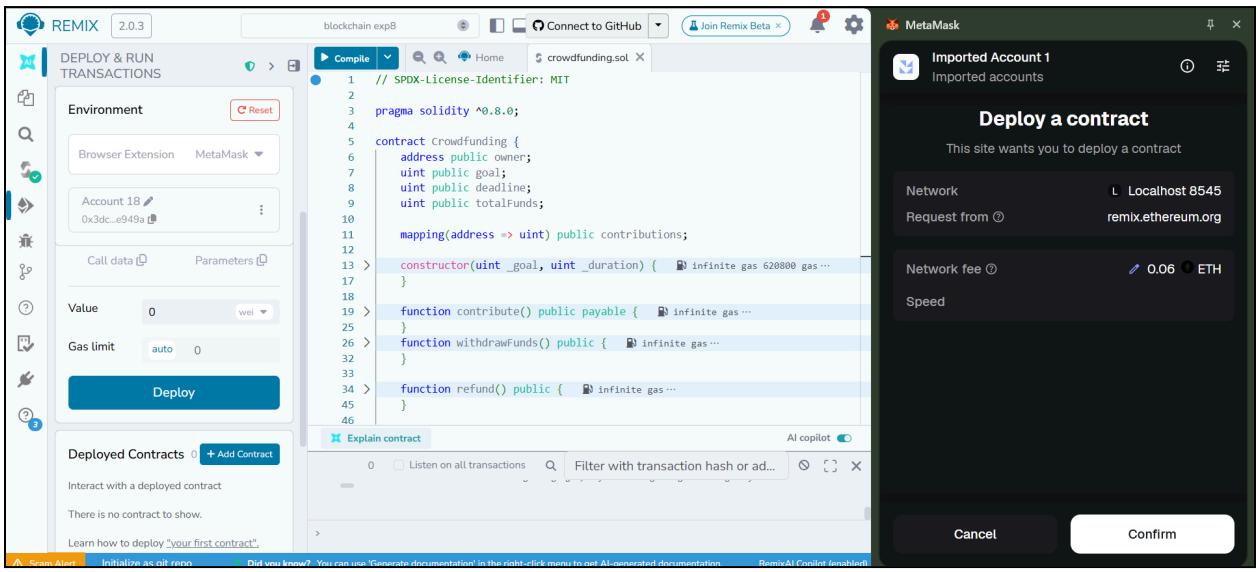
Step 4: In Remix IDE, create a new file **crowdfunding.sol**

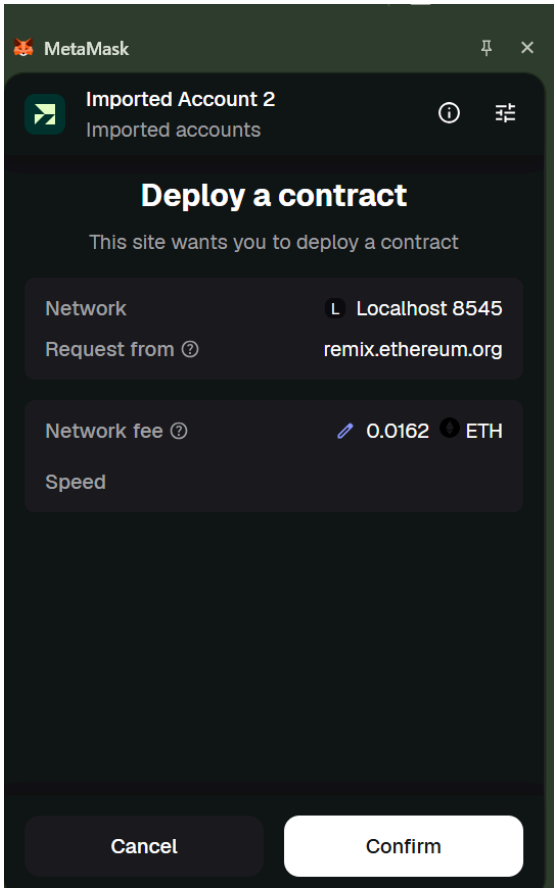
Code:

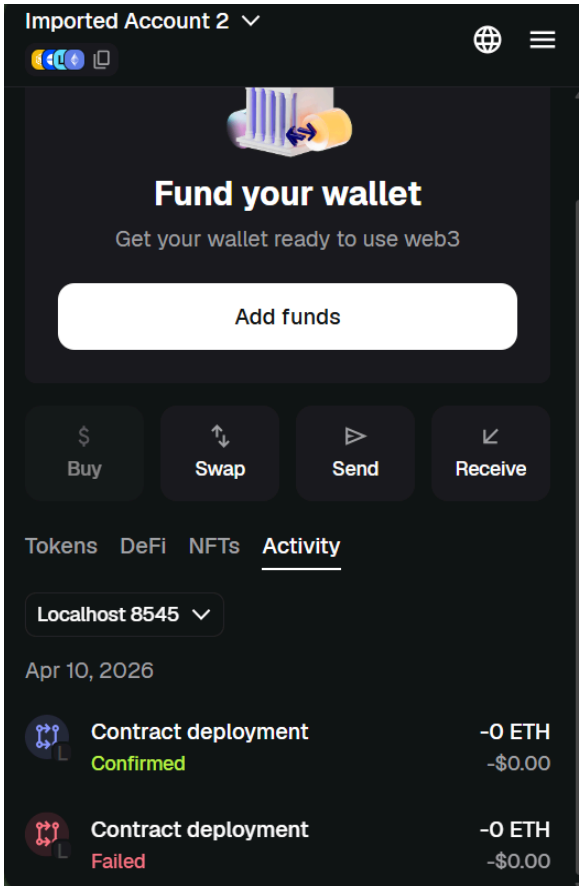
```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

contract Crowdfunding {
    address public owner;
    uint public goal;
    uint public deadline;
    uint public totalFunds;

    mapping(address => uint) public contributions;

    constructor(uint _goal, uint _duration) {
        owner = msg.sender;
        goal = _goal;
        deadline = block.timestamp + _duration;
    }

    function contribute() public payable {
        require(block.timestamp < deadline, "Deadline passed");
        require(msg.value > 0, "Send some ETH");

        contributions[msg.sender] += msg.value;
        totalFunds += msg.value;
    }
    function withdrawFunds() public {
        require(msg.sender == owner, "Only owner");
        require(totalFunds >= goal, "Goal not reached");

        (bool success, ) = payable(owner).call{value: totalFunds}("");
        require(success, "Transfer failed");
    }

    function refund() public {
        require(block.timestamp > deadline, "Not ended");
        require(totalFunds < goal, "Goal was reached");

        uint amount = contributions[msg.sender];
        require(amount > 0, "No contribution");
```

```
        contributions[msg.sender] = 0;

        (bool success, ) = payable(msg.sender).call{value: amount}("");
        require(success, "Refund failed");
    }

    function getBalance() public view returns (uint) {
        return address(this).balance;
    }
}
```

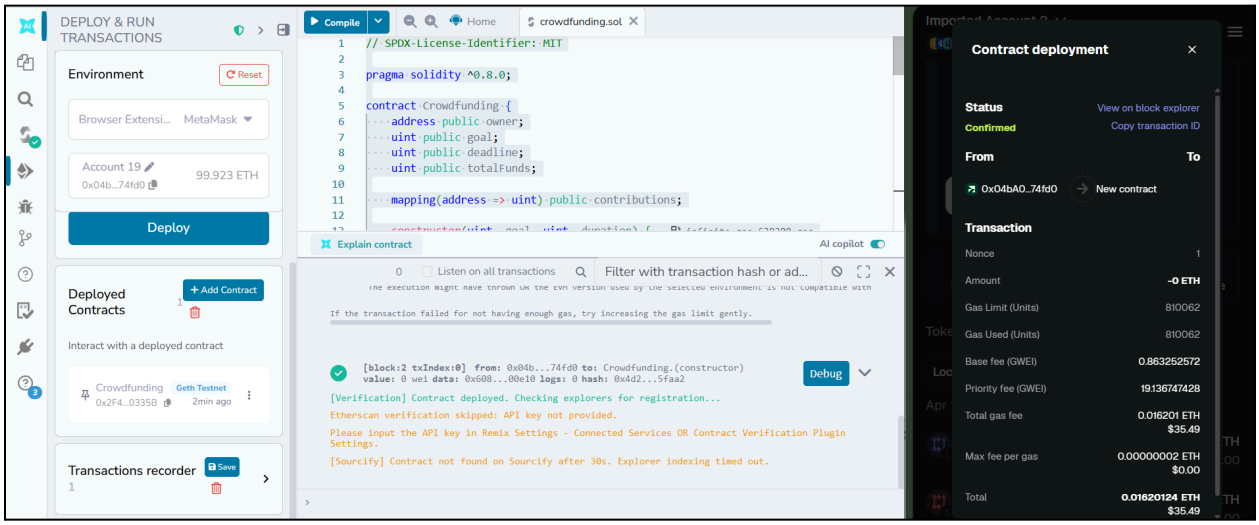
Step 5: Compile and run the contact.
When prompted in metamask, confirm the transaction.



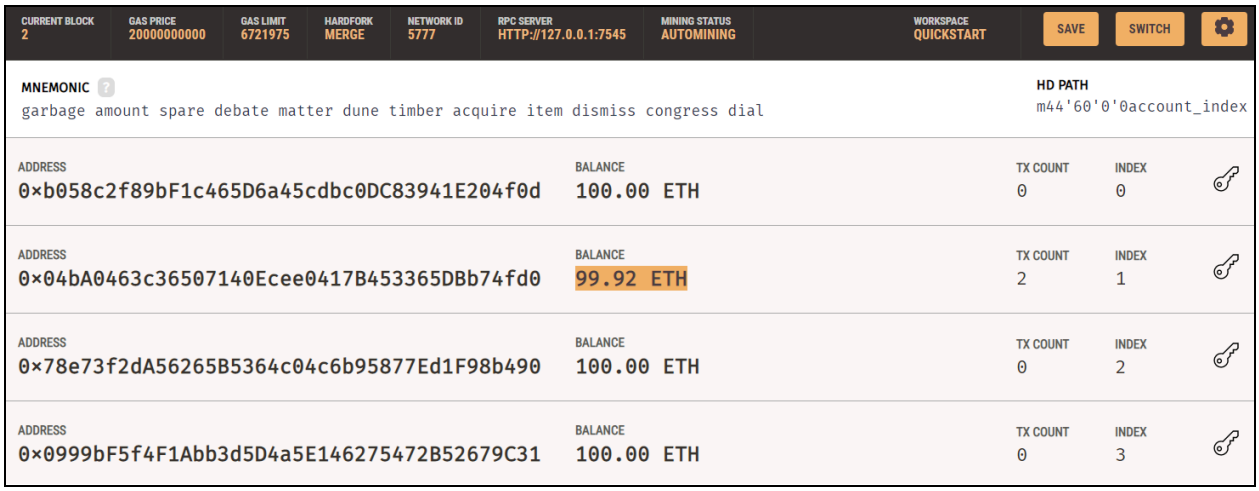




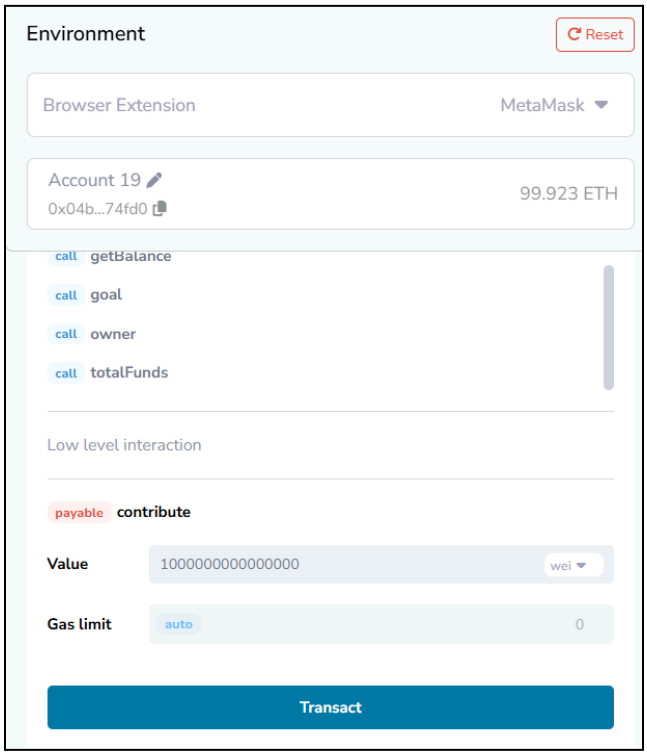
The contract is deployed!

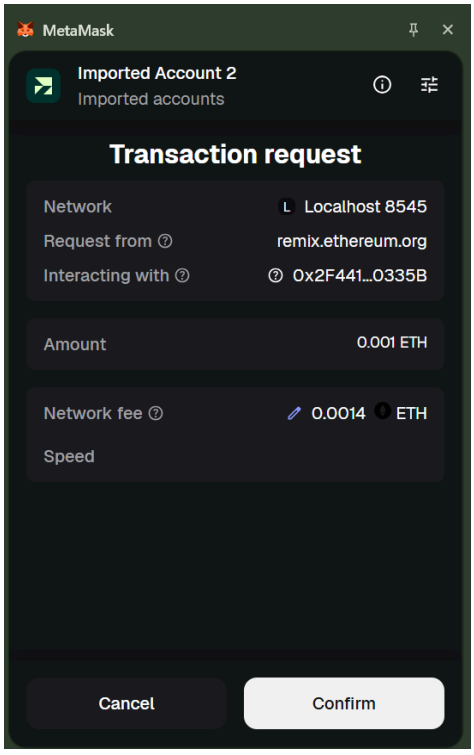


The deducted amount can be seen in ganache app along with transaction count

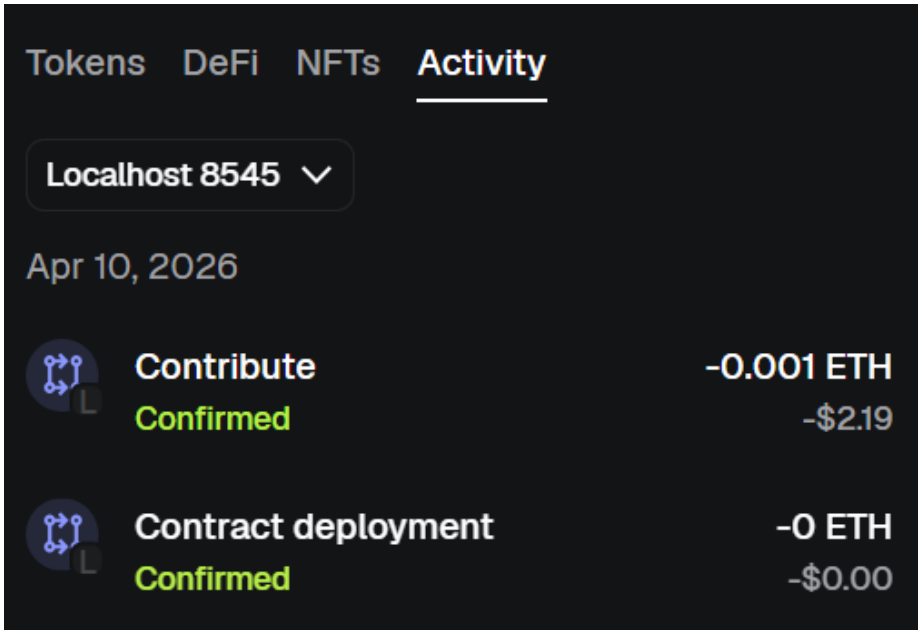


Step 6: Add a new contribution for 0.01 ETH

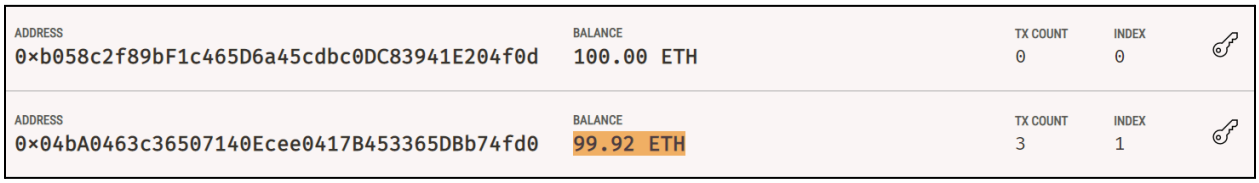




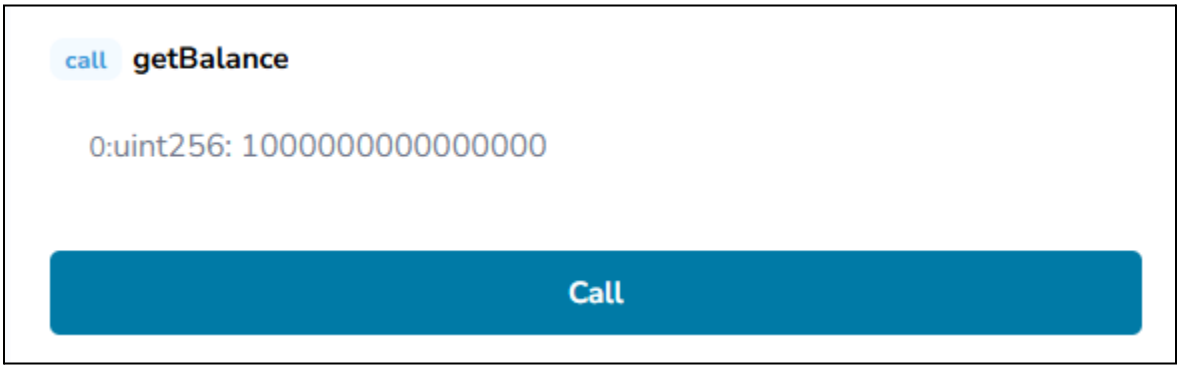
Confirm the transaction in MetaMask.



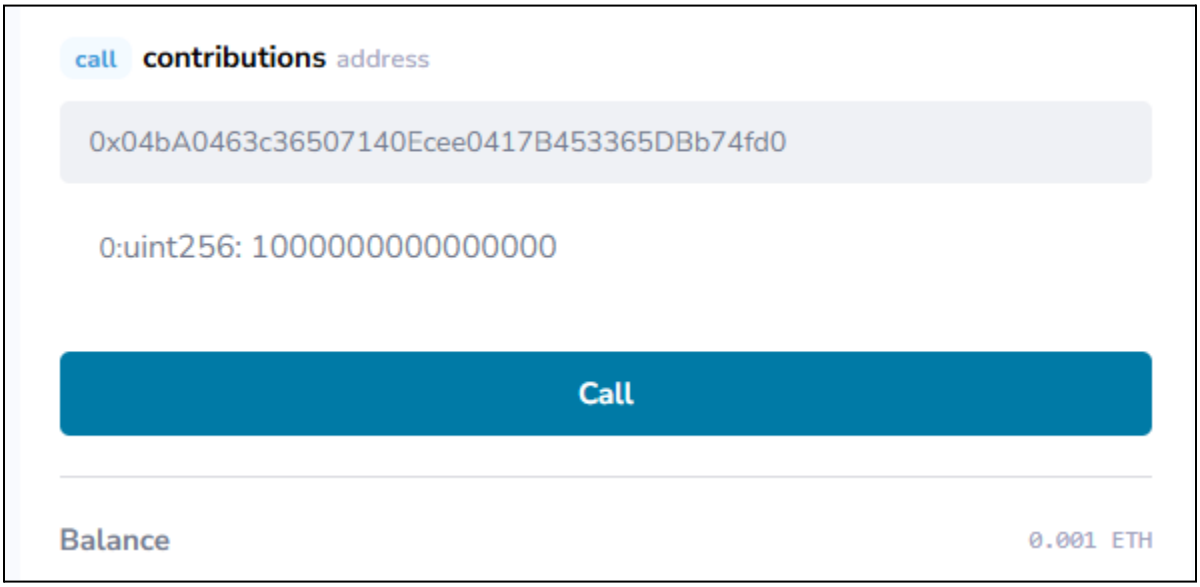
Even in ganache, the transaction count and balance is updated



Step 7: use getBalance to get the total balance



Step 8: Paste wallet address from MetaMask to see all contributions made through that wallet



Conclusion:

In this experiment, a complete local Ethereum development environment was successfully set up by connecting MetaMask to Ganache and deploying a smart contract through Remix IDE. The experiment demonstrated an understanding of configuring network settings, including RPC URLs and Chain IDs, managing cryptocurrency accounts across multiple tools, and connecting wallet providers to development environments. It also involved troubleshooting compatibility issues between different components.

The final deployment was achieved after resolving EVM compiler settings in Remix, which allowed the Crowdfunding smart contract to compile and deploy successfully to the local Ganache network with proper account funding and correct network connectivity across all three tools (Ganache → MetaMask → Remix).